

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4) Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

DATA INTERPRETATION

Code of Conduct:

I Athavan K(192524036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Athavan k

Annexure A

DATA INTERPRETATION

Q1. Intermediate Code Analysis and Optimization Given the three-address code:

$t1 = x + y$ $t2 = t1 * z$ $t3 = t2 / w$ $t4 = t3 - p$

Analyze the sequence of operations and construct the equivalent expression tree. Evaluate how temporary variables improve intermediate code generation and reduce repeated computations.

Q2. Symbol Table and Memory Allocation Given the following symbol table entries:

Variable Data Type Size (Bytes)

p	char	1
q	double	8

r int 4
s float 4

Interpret how the compiler allocates memory for these variables and calculate the total memory required. Analyze how alignment and data types affect memory organization.

Q3. Optimization in Intermediate Code Given the intermediate code:

t1 = m * n t2 = p + q t3 =
m * n t4 = t2 - t3

Trace the execution step-by-step and identify optimization opportunities such as common subexpression elimination and redundant computation removal. Explain how optimization improves execution efficiency.

Q4. Control Flow and Boolean Expression Handling Given the Boolean expression: if (x > y || p < q) Intermediate code: if x > y goto L1 if p < q goto L1 goto L2
L1: ...

Interpret the control flow of the given intermediate representation and explain how shortcircuit evaluation minimizes unnecessary condition checking during execution.

Q5. Backpatching and Flow Control Given:

Instruction	Target
-------------	--------

200: if x==y goto _

201: goto _

Backpatch lists:

- truelist = {200}
- falselist = {201}

Analyze the role of backpatching in intermediate code generation. Demonstrate how target addresses are updated during control flow handling and explain its importance in Boolean expression translation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q1. Intermediate Code Analysis and Optimization

Given the three-address code (TAC):

```
t1 = x + y
t2 = t1 * z
t3 = t2 / w
t4 = t3 - p
```

1. Understanding the Data — Reading the Sequence

The four statements form a linear chain of computation in which every instruction, after the first, consumes the temporary produced by the instruction before it. Tracing the data dependency: t1 depends on {x, y}; t2 depends on {t1, z}; t3 depends on {t2, w}; and t4 depends on {t3, p}. Because each instruction has exactly one operator and at most two operands (the defining property of three-address code), the sequence is a direct linearisation of the source expression:

```
((x + y) * z) / w - p
```

This confirms that the compiler's intermediate-code generator has decomposed a single, deeply nested arithmetic expression into a flat sequence of elementary operations, each introducing a new compiler-generated temporary (t1..t4).

2. Expression Tree Construction

Reversing the dependency chain (innermost operation becomes the deepest leaf-side subtree) yields the following expression tree, where each internal node is an operator and each leaf is either a source variable or a constant:

Expression Tree for $t1=x+y$, $t2=t1*z$, $t3=t2/w$, $t4=t3-p$

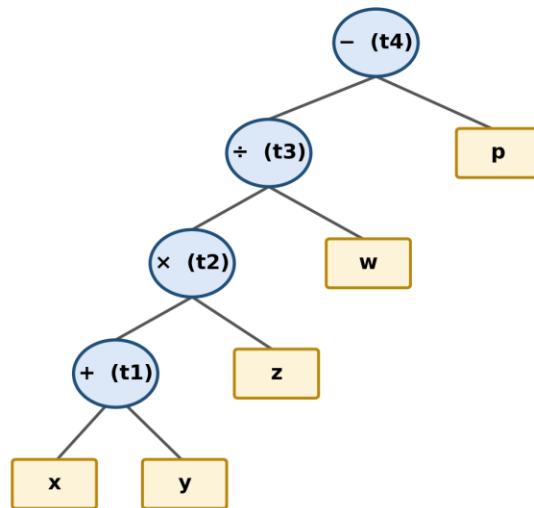


Figure 1: Expression tree equivalent to $t1=x+y$, $t2=t1*z$, $t3=t2/w$, $t4=t3-p$

Reading the tree bottom-up reproduces exactly the given TAC: the leftmost, deepest subtree $(x + y)$ is evaluated first and labelled $t1$; combining it with z under $'*'$ gives $t2$; dividing by w gives $t3$; and finally subtracting p at the root gives $t4$, the final result of the whole expression.

3. Analysis — Role of Temporary Variables

- One operation per instruction: Each temporary stores the result of exactly one operator, so the intermediate representation matches the branching structure of the expression tree node-for-node — this is what makes TAC easy to translate directly into a tree or a DAG.
- Naming intermediate results removes re-evaluation: Once $t1 = x + y$ is computed and stored, every later instruction that needs $(x + y)$ simply reads $t1$ instead of recomputing the addition — this is precisely the mechanism that common-subexpression elimination (Q3) exploits.
- Enables machine-independent optimization: Because temporaries expose every intermediate value explicitly, the optimizer can reorder, fold constants, or eliminate dead code at the TAC level before any target-specific code is generated.
- Simplifies register allocation: Each temporary has a short, well-defined live range (created by one instruction, consumed by the next), which lets the code generator

map temporaries to registers efficiently using liveness analysis, instead of re-deriving values from scratch.

- Facilitates DAG construction: If $t1$'s value were needed again later in the same block, a DAG (Directed Acyclic Graph) representation would let it share the same node rather than being duplicated — collapsing repeated computations to a single evaluation.

4. Conclusion

The four TAC statements are a faithful, one-operator-per-line linearisation of the nested expression $((x+y)*z)/w) - p$, and they correspond exactly to a post-order traversal of the expression tree in Figure 1. By binding every partial result to a named temporary, the compiler decouples expression structure from evaluation order — this is the single design choice that unlocks common-subexpression elimination, safe instruction reordering, precise liveness-based register allocation, and eventual DAG-based redundancy removal. In short, temporaries are not just bookkeeping: they are the mechanism through which intermediate code becomes optimizable.

Q2. Symbol Table and Memory Allocation

Given symbol table entries:

Variable	Data Type	Size (Bytes)
p	char	1
q	double	8
r	int	4
s	float	4

1. Understanding the Data — Reading the Symbol Table

The symbol table records, for every identifier, the attributes the compiler needs to generate correct code: its data type (which determines the operations legal on it) and its size in bytes (which determines how much storage the allocator must reserve). A naive, packed allocation — simply summing the byte column — gives:

Naive total = 1 (p) + 8 (q) + 4 (r) + 4 (s) = 17 bytes

However, real compilers rarely allocate this tightly, because most CPU architectures require multi-byte data types to sit at addresses that are multiples of their own size (their 'natural alignment') for fast, single-cycle memory access.

2. Application of Concepts — Alignment and Padding

- char (p): 1-byte size, 1-byte alignment — can start at any offset.
- double (q): 8-byte size, typically 8-byte alignment — must start at an offset that is a multiple of 8.
- int (r): 4-byte size, 4-byte alignment — must start at a multiple of 4.
- float (s): 4-byte size, 4-byte alignment — must start at a multiple of 4.

Allocating the variables in declaration order (p, q, r, s), the compiler places p at offset 0. The next variable, q (a double), cannot legally start at offset 1, so the compiler inserts 7 bytes of padding to push it to offset 8. r and s then follow immediately, since offset 16 and offset 20 are already multiples of 4.

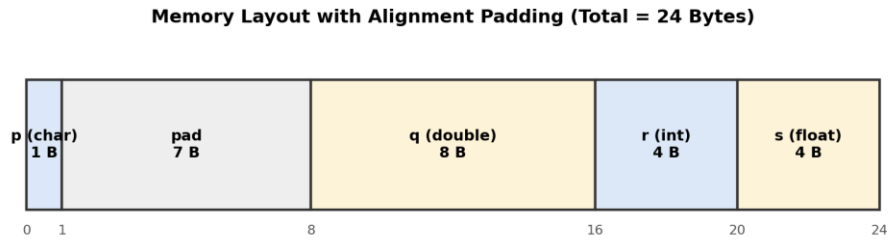


Figure 2: Memory layout for p , q , r , s in declaration order, showing 7 bytes of alignment padding inserted before q

Segment	Offset Range	Size
p (char)	0 – 1	1 B
padding	1 – 8	7 B
q (double)	8 – 16	8 B
r (int)	16 – 20	4 B
s (float)	20 – 24	4 B

Total allocated memory (declaration order, with alignment) = 24 bytes, i.e. 7 bytes more than the packed total of 17 bytes.

3. Analysis — Optimizing the Layout

A compiler's storage allocator can reduce wasted padding by reordering fields from largest to smallest alignment requirement before laying them out — a common struct-packing optimization. Sorting the same four variables as q (8), r (4), s (4), p (1) removes the internal gap entirely:

```

q (double) : offset 0 – 8 (8 B)
r (int)    : offset 8 – 12 (4 B)
s (float)  : offset 12 – 16 (4 B)
p (char)   : offset 16 – 17 (1 B)
trailing pad: offset 17 – 20 (3 B, to align the whole block to 8)
Optimized total = 20 bytes

```


This shows two valid, defensible answers depending on assumptions: 17 bytes if the target ignores alignment (e.g., byte-packed storage or `attribute((packed))` in GCC), 24 bytes if fields are allocated strictly in declared order with natural alignment, and 20 bytes if the compiler is free to reorder fields to minimise padding. All three are derived from the same size column — only the alignment policy differs.

4. Conclusion

Memory allocation for a symbol table is never just 'sum the sizes.' The compiler must also honour each type's alignment constraint, which is why the double variable `q` forces padding bytes to appear before it. Declaration-order allocation costs 24 bytes (17 bytes of data + 7 bytes of padding), while a size-aware reordering strategy recovers most of that waste and brings the total down to 20 bytes. This demonstrates why alignment and data-type ordering are first-class concerns in symbol-table-driven storage allocation, directly affecting both memory footprint and access speed.

Q3. Optimization in Intermediate Code

Given the intermediate code:

```
t1 = m * n
t2 = p + q
t3 = m * n
t4 = t2 - t3
```

1. Understanding the Data — Step-by-Step Trace

Step	Instruction	Observation
1	$t1 = m * n$	Computes $m*n$ for the first time; result stored in $t1$.
2	$t2 = p + q$	Independent computation; no operands overlap with $t1$.
3	$t3 = m * n$	Identical expression to Step 1 — m and n are unchanged since Step 1, so $t3$ is guaranteed to equal $t1$.
4	$t4 = t2 - t3$	Uses the redundant value $t3$; could equally use $t1$.

2. Analysis — Identifying the Redundancy

Step 3 recomputes $m * n$ even though Step 1 already produced this exact value in $t1$, and neither m nor n is redefined in between. This is a textbook case for common subexpression elimination (CSE): whenever two instructions compute the same expression from operands that have not changed in the interim, the later computation is provably redundant and can be replaced by a reference to the earlier result.

3. Application of Concepts — Applying CSE

Replacing $t3$'s re-computation with the already-available $t1$ removes one multiplication instruction entirely:

```
t1 = m * n
t2 = p + q
t4 = t2 - t1    // t3 eliminated; t1 reused directly
```

The optimized sequence has three instructions instead of four — a 25% reduction in instruction count for this block — with zero change in the final value of t_4 , since t_3 and t_1 were mathematically identical.

4. Analysis — Why This Improves Execution Efficiency

- Fewer arithmetic operations: one multiplication is completely removed from the execution path, directly reducing CPU cycles.
- Reduced register / memory pressure: t_3 no longer needs a register or stack slot, freeing resources for other live variables.
- Shorter dependency chain: t_4 can be computed as soon as t_1 and t_2 are ready, without waiting on a redundant recomputation.
- Compounding benefit in loops: if this code executed inside a loop, eliminating one redundant multiplication per iteration could produce a significant cumulative speed-up.

5. Conclusion

Tracing the four instructions exposes that $t_3 = m * n$ is a common subexpression of $t_1 = m * n$, since m and n are not modified between the two statements. Eliminating this redundancy through CSE reduces the code from four instructions to three, lowers register demand, and shortens the critical path to computing t_4 — a direct, measurable improvement in both code size and execution efficiency, achieved purely through intermediate-code-level optimization, before any target-machine code is generated.

Q4. Control Flow and Boolean Expression Handling

Given the Boolean expression and its intermediate code:

```
if (x > y || p < q)
```

```
  if x > y goto L1
```

```
  if p < q goto L1
```

```
  goto L2
```

```
L1: ...
```

1. Understanding the Data — Interpreting the Control Flow

The intermediate code translates a single OR expression into a chain of conditional jumps rather than computing a Boolean value and testing it afterward. Execution proceeds as follows: first, $x > y$ is evaluated; if true, control jumps directly to L1 (the true branch) without ever evaluating $p < q$. Only if $x > y$ is false does execution fall through to test $p < q$; if that is true, control again reaches L1. If both conditions are false, control falls through to goto L2, the false branch.

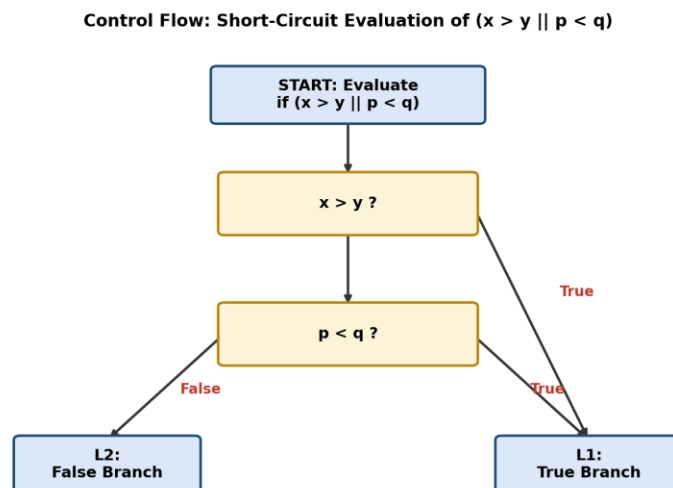


Figure 3: Control-flow graph for $\text{if } (x > y \parallel p < q)$, showing short-circuit evaluation

2. Analysis — Why This Is Short-Circuit Evaluation

- For OR ($\parallel$), if the first operand is true, the overall expression is already guaranteed true — so $p < q$ is never evaluated. This is exactly what the jump-to-L1-on-true-of-the-first-condition achieves.
- The second condition ($p < q$) is only reached on the path where the first condition was false — mirroring the semantics of $\parallel$ precisely, since only in that case can the second operand still change the outcome.
- goto L2 is placed only after both checks fail, correctly representing the case where the whole disjunction evaluates to false.

3. Application of Concepts — Benefit of Short-Circuiting

- Avoids unnecessary evaluation: expensive or side-effecting sub-conditions (e.g., a function call in $p < q$) are skipped whenever the outcome is already determined.
- Fewer runtime instructions on the common path: when $x > y$ is true, the code executes only one comparison and one jump instead of evaluating both conditions and combining them with a logical OR.
- Natural fit for jump-based intermediate code: representing Boolean expressions as control flow (rather than as computed 0/1 values) lets the code generator emit compact conditional-jump sequences directly usable by the target machine, avoiding a separate Boolean-to-branch conversion step later.
- Prevents unsafe evaluation: in languages where operands may be undefined or cause errors (e.g., a null-pointer check ORed with a dereference), short-circuiting guarantees the unsafe operand is never evaluated once the result is already known.

4. Conclusion

The given three-instruction sequence is a direct, efficient encoding of short-circuit OR semantics: it tests $x > y$ first and jumps immediately on success, only falling through to evaluate $p < q$ when necessary, and only reaches the false label L2 when both tests fail. This jump-chaining strategy minimizes unnecessary condition checking, reduces instruction-path length on the common case, and safely defers or skips evaluation of the second operand — making it both a correctness-preserving and a performance-conscious translation of Boolean control flow.

Q5. Backpatching and Flow Control

Given:

Instruction	Target
200: if x==y goto _	(unresolved)
201: goto _	(unresolved)

```
truelist = {200}  
falselist = {201}
```

1. Understanding the Data — What the Lists Represent

Instructions 200 and 201 are conditional/unconditional jumps whose target addresses are deliberately left blank ('_') because, at the moment they are generated, the compiler does not yet know where the true-branch code or the false-branch code will begin. `truelist = {200}` records that instruction 200 must eventually jump to wherever the 'true' continuation of this Boolean expression is placed; `falselist = {201}` records the same obligation for instruction 201, pointing to the eventual 'false' continuation.

2. Application of Concepts — The Backpatching Mechanism

Backpatching solves a classic one-pass-compiler problem: target addresses for forward jumps are not known until later code has been generated. Rather than generating code in two passes, the compiler generates jump instructions with empty targets, tracks their instruction numbers in lists (`truelist`, `falselist`), and fills in ('patches') the blank target field only once the real address becomes known — typically when the next statement's label is finally emitted.

Backpatching: Resolving Jump Targets After Address is Known

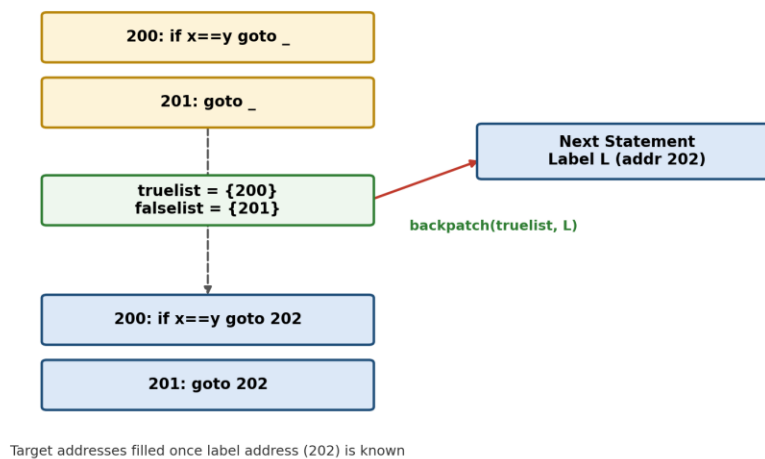


Figure 4: Backpatching $truelist = \{200\}$ once the target label's address (202) becomes known

Three standard helper functions implement this in a syntax-directed translation scheme:

- `makelist(i)`: creates a new list containing only instruction `i`, used when a single unresolved jump is first generated.
- `merge(list1, list2)`: concatenates two lists into one, used when control can reach the same later point via more than one jump (e.g., combining true-lists across an OR or AND expression).
- `backpatch(list, target)`: inserts the address 'target' into the target field of every instruction whose number appears in 'list' — this is the step that actually resolves the previously blank jumps.

3. Analysis — Applying It to This Example

Suppose the next statement generated after instruction 201 begins at address 202. When the parser reaches that point and finally knows this address, it calls `backpatch(truelist, 202)`, which locates instruction 200 (the only entry in `truelist`) and rewrites its target from `'_'` to 202:

```
200: if x==y goto 202
```

Separately, if the false branch's code is emitted starting at some other known address (or falls through and is patched at the appropriate later point), a call such as `backpatch(falselist, 202)` similarly resolves instruction 201:

201: goto 202

Both jumps are now fully resolved, and the intermediate code is executable without ever having required a second scan of the source or a symbol-address forward-reference table.

4. Conclusion

Backpatching allows a compiler to emit conditional and unconditional jumps in a single left-to-right pass even though their true destinations are not yet known, by deferring address resolution through truelist/falselist bookkeeping and a later `backpatch(list, target)` call. In this example, instruction 200 (from truelist) and instruction 201 (from falselist) both start with blank targets and are correctly patched to address 202 once that address is discovered — demonstrating exactly why backpatching is indispensable for translating Boolean expressions and control statements (`if`, `while`, `for`) efficiently in one pass, without the overhead of a separate backfilling phase or intermediate symbolic labels in the final code.